

1. What is Apache Airflow?

Apache Airflow is an **open-source workflow orchestration platform** designed to **author, schedule, and monitor workflows programmatically**.

It is widely used in modern data engineering to manage **complex data pipelines**, ensuring tasks are executed in the correct order with reliability, scalability, and visibility.

1.1 Definition

Apache Airflow can be formally defined as:

A platform that allows users to **define workflows as code (using Python)**, **schedule their execution**, and **monitor their progress through a user-friendly interface**.

1.2 Purpose

The primary purpose of Apache Airflow is to:

- Automate **data pipeline workflows**
 - Manage **task dependencies**
 - Schedule and execute tasks at specified intervals
 - Provide **monitoring and logging capabilities**
 - Ensure **fault tolerance** through retries and alerts
-

1.3 Key Characteristics

a) Workflow as Code

Airflow allows workflows to be written in **Python scripts**, making them:

- Dynamic
- Version-controllable
- Easily maintainable

b) Directed Acyclic Graph (DAG) Based

Workflows are structured as **Directed Acyclic Graphs (DAGs)**, where:

- Nodes represent tasks
- Edges represent dependencies
- No cyclic dependencies are allowed

c) Scalable Execution

Airflow supports multiple execution modes:

- Sequential (single-threaded)
- Parallel (multi-threaded)
- Distributed (using Celery or Kubernetes)

d) Rich User Interface

Airflow provides a **web-based UI** that allows users to:

- Visualize workflows
 - Track task status
 - View logs
 - Trigger or retry tasks manually
-

1.4 Core Functionalities

Apache Airflow provides the following core functionalities:

1. Workflow Definition

- Define pipelines using Python scripts
- Specify task dependencies and execution order

2. Scheduling

- Supports cron-like scheduling
- Enables periodic or event-based execution

3. Task Orchestration

- Ensures tasks run in the correct sequence
- Handles dependencies automatically

4. Monitoring and Logging

- Provides detailed logs for each task
- Tracks execution history

5. Error Handling

- Supports retries on failure
 - Allows alerting mechanisms (e.g., email notifications)
-

1.5 Typical Workflow Example

A common use case of Airflow in a data engineering pipeline:

1. Extract data from a source (e.g., API or database)
2. Transform the data (cleaning, aggregation)
3. Load the data into a data warehouse

Airflow ensures:

- Each step runs in sequence
 - Failures are handled properly
 - The pipeline runs automatically at scheduled intervals
-

1.6 Why Apache Airflow is Used

Organizations adopt Apache Airflow because it:

- Provides **end-to-end workflow orchestration**
 - Supports **complex dependency management**
 - Integrates with various tools (e.g., Spark, Hadoop, cloud services)
 - Enables **automation of repetitive processes**
 - Improves **reliability and observability** of data pipelines
-

1.7 Industry Applications

Apache Airflow is commonly used in:

- **Data Engineering** – ETL/ELT pipelines
- **Machine Learning** – Model training and deployment workflows
- **Data Warehousing** – Scheduled data ingestion
- **Cloud Operations** – Orchestrating multi-service workflows

2. Advantages of Apache Airflow

Apache Airflow provides several advantages that make it a preferred choice for workflow orchestration in data engineering and enterprise environments.

2.1 Workflow as Code (Flexibility and Maintainability)

- Workflows in Airflow are defined using **Python scripts**, enabling:
 - Dynamic pipeline creation

- Use of programming constructs (loops, conditions, functions)
 - Easy version control using tools like Git
 - This approach ensures better **maintainability and scalability** compared to static workflow tools.
-

2.2 Scalable and Distributed Execution

- Airflow supports multiple executors, allowing it to scale from:
 - Single-node execution to
 - Distributed systems using Celery or Kubernetes
 - This ensures:
 - Efficient handling of large workloads
 - Parallel execution of tasks
 - High performance in enterprise environments
-

2.3 Robust Scheduling Capabilities

- Provides **cron-like scheduling** for workflows
 - Supports:
 - Time-based scheduling (hourly, daily, etc.)
 - Event-driven execution
 - Backfilling (running past workflows)
 - Enables automation of recurring tasks with precision and reliability
-

2.4 Dependency Management

- Airflow allows explicit definition of **task dependencies** using DAGs
 - Ensures:
 - Correct execution order
 - No task runs before its dependencies are met
 - Clear visualization of workflow structure
-

2.5 Rich User Interface (UI)

- Airflow provides an intuitive web interface to:
 - Monitor workflows in real time
 - Visualize DAGs and task relationships
 - Access logs for debugging
 - Trigger or retry tasks manually
 - Improves **observability and operational control**
-

2.6 Fault Tolerance and Reliability

- Built-in mechanisms for:
 - Automatic retries on task failure
 - Alerting (email, notifications)
 - Logging of execution details
 - Ensures workflows are **resilient and reliable**
-

2.7 Extensibility and Integration

- Airflow supports integration with a wide range of tools and platforms:
 - Apache Spark
 - Hadoop ecosystem
 - Cloud services (AWS, Azure, GCP)
 - Databases and APIs
 - Custom operators and plugins can be developed to extend functionality
-

2.8 Strong Community and Open Source Support

- Being open-source, Airflow benefits from:
 - Continuous improvements
 - Community-contributed plugins and features
 - Extensive documentation and support
-

2.9 Monitoring and Logging

- Detailed logs are maintained for each task execution

- Provides:
 - Historical tracking of workflows
 - Easy debugging of failures
 - Performance analysis
-

2.10 Improved Productivity and Automation

- Automates repetitive and manual processes
- Reduces human intervention and errors
- Enables teams to focus on higher-level tasks

3. Apache Airflow Architecture

The architecture of **Apache Airflow** is designed to support **scalable, distributed, and reliable workflow execution**. It follows a modular structure where different components interact to manage the lifecycle of workflows (DAGs), from scheduling to execution and monitoring.

3.1 Overview of Architecture

Apache Airflow consists of the following core components:

1. **Web Server**
2. **Scheduler**
3. **Executor**
4. **Workers**
5. **Metadata Database**
6. **DAG Files (Code Repository)**

Each component has a specific responsibility and works together to ensure smooth orchestration of workflows.

3.2 Core Components

3.2.1 Web Server

- Provides a **user interface (UI)** for interacting with Airflow
- Allows users to:
 - View DAGs and their structures

- Monitor task execution
 - Access logs
 - Trigger or retry workflows
 - Acts as the **visual layer** of the system
-

3.2.2 Scheduler

- The **central component (brain)** of Airflow
 - Responsible for:
 - Parsing DAG files
 - Determining task dependencies
 - Scheduling tasks based on defined intervals and conditions
 - Continuously checks:
 - Which tasks are ready to run
 - Which tasks are completed or failed
-

3.2.3 Executor

- Responsible for **executing tasks**
 - Receives tasks from the scheduler and assigns them for execution
 - Common types of executors:
 - **Sequential Executor** – runs one task at a time
 - **Local Executor** – runs tasks in parallel on a single machine
 - **Celery Executor** – distributed execution across multiple worker nodes
 - **Kubernetes Executor** – runs tasks in Kubernetes pods
-

3.2.4 Workers

- Workers are responsible for **actually running the tasks**
- Used in distributed setups (e.g., Celery Executor)
- Each worker:
 - Picks up tasks from a queue
 - Executes them

- Sends status updates back to the system
-

3.2.5 Metadata Database

- Stores all **state and metadata information**, including:
 - DAG definitions (parsed information)
 - Task states (running, success, failed)
 - Execution logs
 - Scheduling information
 - Common databases used:
 - PostgreSQL
 - MySQL
-

3.2.6 DAG Files (Code Repository)

- DAGs are defined as **Python scripts**
 - Stored in a directory accessible by the scheduler
 - The scheduler:
 - Continuously scans this directory
 - Parses DAG definitions
 - Updates the metadata database
-

3.3 Workflow Execution Flow

The execution of a workflow in Airflow follows these steps:

1. **DAG Creation**
 - User defines a DAG using Python
2. **DAG Parsing**
 - Scheduler scans DAG files and identifies tasks and dependencies
3. **Task Scheduling**
 - Scheduler determines which tasks are ready to execute
4. **Task Submission**
 - Tasks are sent to the executor

5. Task Execution

- Executor assigns tasks to workers (or runs locally)

6. State Update

- Task status is updated in the metadata database

7. Monitoring

- Users track progress via the web server UI
-

3.4 Architecture Modes

Apache Airflow can be deployed in different configurations:

a) Standalone Mode

- All components run on a single machine
- Suitable for development and testing

b) Distributed Mode

- Components run on multiple machines
 - Uses Celery or Kubernetes Executor
 - Suitable for production environments
-

3.5 Key Architectural Features

- **Modularity** – Components are loosely coupled
- **Scalability** – Supports distributed execution
- **Fault Tolerance** – Handles failures with retries and logging
- **Extensibility** – Custom operators and plugins can be added

4. What is a DAG and What are Executors

In **Apache Airflow**, two of the most fundamental concepts are **DAGs (Directed Acyclic Graphs)** and **Executors**. Together, they define **how workflows are structured and how tasks are executed**.

4.1 What is a DAG (Directed Acyclic Graph)?

A **DAG** is the core building block of Apache Airflow. It represents a **workflow as a collection of tasks with defined dependencies**.

4.1.1 Definition

A Directed Acyclic Graph (DAG) is a graph where:

- **Directed** → Tasks have a defined direction (execution flow)
 - **Acyclic** → No loops or cycles are allowed
 - **Graph** → Consists of nodes (tasks) and edges (dependencies)
-

4.1.2 Key Components of a DAG

1. Tasks (Nodes)

- Individual units of work
- Example: Extract data, transform data, load data

2. Dependencies (Edges)

- Define execution order
 - Example: Task B runs only after Task A completes
-

4.1.3 Characteristics of a DAG

- Ensures **sequential and logical execution**
 - Prevents **infinite loops**
 - Supports **parallel execution** where dependencies allow
 - Fully defined using **Python code**
-

4.1.4 Example of a DAG Structure

Task A → Task B → Task C

↓

Task D

- Task B depends on Task A
 - Task C and Task D depend on Task B
 - Tasks C and D can run in parallel
-

4.1.5 Sample DAG Code

```
from airflow import DAG
from airflow.operators.dummy import DummyOperator
from datetime import datetime
```

```
with DAG(
    dag_id="example_dag",
    start_date=datetime(2024, 1, 1),
    schedule_interval="@daily"
) as dag:

    task1 = DummyOperator(task_id="start")
    task2 = DummyOperator(task_id="process")
    task3 = DummyOperator(task_id="end")

    task1 >> task2 >> task3
```

4.1.6 Importance of DAGs

- Provides **clear workflow structure**
 - Helps manage **complex dependencies**
 - Enables **visualization and monitoring**
 - Ensures **reliable execution of pipelines**
-

4.2 What are Executors in Airflow?

An **Executor** in Apache Airflow is the component responsible for **running the tasks defined in a DAG**.

4.2.1 Definition

An Executor determines **how and where tasks are executed** after being scheduled.

It acts as a bridge between the **Scheduler** and the **actual execution environment**.

4.2.2 Role of Executors

- Receives tasks from the scheduler
 - Decides execution strategy
 - Distributes tasks to workers or executes them locally
 - Manages parallelism and resource usage
-

4.2.3 Types of Executors

1. Sequential Executor

- Default executor
 - Executes **one task at a time**
 - Suitable for:
 - Testing
 - Development
-

2. Local Executor

- Executes multiple tasks in parallel on a **single machine**
 - Uses multiprocessing
 - Suitable for:
 - Small to medium workloads
-

3. Celery Executor

- Distributed executor using **Celery workers**
 - Tasks are queued and executed across multiple worker nodes
 - Suitable for:
 - Large-scale production systems
-

4. Kubernetes Executor

- Executes each task in a **separate Kubernetes pod**
 - Provides:
 - High scalability
 - Isolation
 - Suitable for:
 - Cloud-native environments
-

4.2.4 Executor Workflow

1. Scheduler identifies tasks ready to run
2. Tasks are sent to the executor
3. Executor assigns tasks to:

- Local processes OR
 - Worker nodes OR
 - Kubernetes pods
4. Tasks are executed
 5. Status is updated in the metadata database

4.2.5 Choosing the Right Executor

Executor Type	Use Case
Sequential	Development and testing
Local	Moderate workloads
Celery	Distributed large-scale systems
Kubernetes	Cloud-native, highly scalable